

Capítulo 1

El Lenguaje de Dominio Específico

1.1. Visión general

El lenguaje de dominio específico (DSL) desarrollado en este trabajo permite describir problemas de planificación combinatoria mediante tres conceptos fundamentales: *entidades* del dominio, *consultas* que calculan métricas sobre un horario, y *restricciones* que expresan las reglas que todo horario válido debe satisfacer.

El DSL está implementado como un conjunto de macros de Common Lisp. Cada forma del lenguaje construye, en tiempo de carga, un *árbol de sintaxis abstracta* (AST) cuyos nodos son instancias de clases CLOS. Ese árbol puede recorrerse posteriormente para generar código Python que evalúa la calidad de un horario y lo optimiza mediante una metaheurística de búsqueda local.

La figura siguiente muestra la jerarquía de clases que compone el AST. Las siguientes secciones describen cada grupo de nodos con sus atributos y su papel en el lenguaje, intercalando ejemplos extraídos del caso de uso de planificación de defensas de tesis.

1.2. Nodos base y representación de entidades

Todo nodo del árbol es instancia, directa o indirecta, de `nodo-ast`. Las entidades del dominio se representan mediante dos familias de clases: las referencias genéricas, que nombran un tipo de entidad, y las referencias específicas, que son instancias concretas con atributos propios.

Cuando el usuario escribe `(def-entidad profesor (tiene-nombre) (grado))`, la macro expande esa forma en tres definiciones: la subclase específica `REF-ESPECIFICA-A-PROFESOR`

Cuadro 1.1: Nodos base y de representación de entidades.

Nodo	Atributos	Papel
nodo-ast	(ninguno)	Clase base absoluta de todos los nodos del árbol. Sirve de raíz común para despacho polimórfico.
ref-gen-entidad	nombre-entidad	Referencia genérica a un tipo de entidad del dominio. El atributo guarda el símbolo con el nombre del tipo (p.ej. profesor).
ref-especifica-ent	(heredados del tipo concreto)	Clase base para las instancias concretas de una entidad. Cada tipo de entidad genera una subclase con sus propios atributos.

(con slots **nombre** y **grado**), la subclase genérica **REF-GEN-A-PROFESOR**, y la variable global **profesor**, ligada a una instancia de la clase genérica.

Listing 1.1: Definición de entidades del dominio.

```
;; mixin reutilizable
(defclass* tiene-nombre () (nombre))

;; entidades simples
(def-entidad profesor (tiene-nombre) (grado))
(def-entidad estudiante (tiene-nombre))
(def-entidad local (tiene-nombre))

;; entidad enumerada (valores fijos)
(def-entidad grado () (id))
(def-grado grado-lic "Lic")
(def-grado grado-msc "Msc")
(def-grado grado-dr "Dr")
```

1.3. Estructuras de datos: combinaciones y listas

Además de las entidades atómicas, el lenguaje dispone de dos nodos para agrupar valores: las *combinaciones de entidades* y las *listas*.

Cuadro 1.2: Nodos de estructuras de datos.

Nodo	Atributos	Papel
comb-entidades	comb-entidades-lista	Representa un grupo heterogéneo de entidades relacionadas (una <i>combinación</i>). Se usa para modelar la entidad principal del problema de horario (p.ej. una asignación).
lista	lista-elementos	Colección homogénea de valores. Sirve como contenedor en operaciones de comprobación de conjuntos y en dominios de iteración.

La macro `def-comb` es azúcar sintáctico sobre `def-entidad`: genera una entidad sin mixins cuyos componentes son referencias a otras entidades del dominio.

Listing 1.2: Definición de la combinación central del problema.

```
;; entidades participantes
(def-entidad tribunal () (estudiante tutor oponente presidente vocal s
(def-entidad fecha    () (dia mes))
(def-entidad momento () (hora))

;; combinacion: una asignacion agrupa tribunal, fecha, momento y local
(def-comb asignacion (tribunal fecha momento local))
```

Cada instancia de `asignacion` es un nodo `comb-entidades` cuyos cuatro componentes son referencias específicas a sus entidades constituyentes.

1.4. Acceso a datos

Para leer el valor de un atributo o de un componente de una entidad se generan nodos de acceso que preservan, en el árbol, la cadena de navegación

hasta el dato final.

Cuadro 1.3: Nodos de acceso a datos.

Nodo	Atributos	Papel
<code>acceso-a-atributo-de-entidad</code>	<code>acceso-atributo</code> <code>acceso-entidad</code>	Representa la lectura de un atributo de una entidad. <code>acceso-atributo</code> es el símbolo del slot; <code>acceso-entidad</code> es el nodo que produce la entidad sobre la que se accede.
<code>acceso-a-elemento-de-combinacion</code>	<code>acceso-comb-entidad</code> <code>acceso-comb-origen</code>	Extrae un componente de una combinación. <code>acceso-comb-entidad</code> identifica el componente deseado; <code>acceso-comb-origen</code> es la combinación de origen.

Cuando se escribe `(tutor (tribunal asig))`, la macro genera un `acceso-a-atributo-de-entidad` cuyo atributo es `tutor` y cuya entidad es a su vez un `acceso-a-elemento-de-combinacion` que extrae el componente `tribunal` de `asig`. El generador de código traduce esta cadena en la expresión Python `asig.tribunal.tutor`.

Listing 1.3: Acceso encadenado a atributos de una combinacion.

```
;; Leer el tutor de la asignacion asig:
(tutor (tribunal asig)) ; -> asig.tribunal.tutor (en Python)

;; Leer la hora del momento de otra asignacion:
(hora (momento asig)) ; -> asig.momento.hora
```

1.5. Consultas

El nodo `consulta-simple` es el bloque fundamental de cómputo del lenguaje. Representa una operación de tipo *comprensión*: itera sobre uno o varios dominios, filtra los elementos que satisfacen una condición y acumula un resultado.

La macro `def-consulta` tiene dos modos de uso. Sin la clave `:itera-sobre` define una función reutilizable que recibe variables AST y devuelve un nodo condición. Con `:itera-sobre` genera una variable global ligada a un nodo `consulta-simple` listo para ser evaluado o generado.

Cuadro 1.4: Nodo de consulta.

Nodo	Atributos	Papel
consulta-simple	consulta-args consulta-var-iter consulta-dominio consulta-comprobacion consulta-operacion consulta-retorno	Consulta parametrizable con iteración sobre dominios, filtrado por condición y acumulación de un resultado. consulta-args son parámetros externos adicionales; consulta-var-iter y consulta-dominio definen los bucles de iteración; consulta-comprobacion es el predicado de filtrado; consulta-operacion indica la forma de acumulación (suma , contar-distintos-dias , etc.); consulta-retorno es el símbolo que se acumula.

Listing 1.4: Consulta que verifica si un profesor pertenece a un tribunal y consulta que cuenta conflictos de horario.

```
;; Funcion auxiliar: devuelve un nodo condicion (sin :itera-sobre)
(def-consulta profesor-en-tribunal (prof asig)
  :comprueba
  (def-op-or (def-op-igual (tutor (tribunal asig)) prof)
    (def-op-igual (oponente (tribunal asig)) prof)
    (def-op-igual (presidente (tribunal asig)) prof)
    (def-op-igual (vocal (tribunal asig)) prof)
    (def-op-igual (secretario (tribunal asig)) prof)))

;; Consulta completa: itera sobre profesores y pares de asignaciones
(def-consulta conflictos-de-horario ()
  :itera-sobre (p profesor) (a1 asignacion) (a2 asignacion)
  :comprueba
  (def-op-and (profesor-en-tribunal p a1)
    (profesor-en-tribunal p a2)
    (def-op-igual (fecha a1) (fecha a2))
    (def-op-igual (momento a1) (momento a2))
    (def-op-distinto (tribunal a1) (tribunal a2))))
```

```
:operacion suma
:devuelve 1)
```

En el ejemplo, `conflictos-de-horario` itera sobre tres dominios (`profesor`, `asignacion`×`asignacion`), filtra por la condición compuesta y acumula la suma de unos; el generador de código traduce todo esto en tres bucles `for` anidados con un `if` interior.

1.6. Cuantificadores de restricción

Las restricciones del problema se expresan mediante cuantificadores que envuelven una comprobación (posiblemente una consulta) e indican cómo debe interpretarse su resultado: como restricción dura, blanda u objetivo de optimización.

Listing 1.5: Restriccion dura y objetivo de minimizacion.

```
;; Restriccion dura: ningun profesor puede estar en dos defensas al mi
;; nppq: "No Puede Pasar Que" conflictos-de-horario sea mayor que 0.
(defvar restriccion-sin-conflictos-de-horario
 (def-nppq (def-op-mayor conflictos-de-horario 0)))

;; Consulta auxiliar: dias distintos que asiste un profesor
(def-consulta dias-asistidos (prof)
 :itera-sobre (asig asignacion)
 :comprueba (profesor-en-tribunal prof asig)
 :operacion contar-distintos-dias
 :devuelve n-dias)

;; Objetivo blando: minimizar la carga del profesor que mas dias asiste
(defvar restriccion-minimizar-carga-maxima
 (def-minimizar dias-asistidos profesor))
```

1.7. Comprobaciones de conjuntos

El lenguaje incluye dos nodos especializados para expresar condiciones sobre colecciones de valores sin necesidad de construir bucles explícitos.

Cuadro 1.5: Nodo base y subclases de cuantificador.

Nodo	Atributos	Papel
cuantificador-restriccion	cuanti-operador cuanti-comprobacion cuanti-elemento	Clase base de todos los cuantificadores. cuanti-comprobacion es el nodo condición o consulta que se evalúa; cuanti-elemento es el dominio de referencia en los cuantificadores de optimización.
nppq	(heredados)	<i>No Puede Pasar Que</i> . Restricción dura: la comprobación no puede ser verdadera. Genera peso alto en la función objetivo.
tppq	(heredados)	<i>Todo Que Pasa Que</i> . Restricción dura: la comprobación debe ser verdadera.
tdpq	(heredados)	<i>Todos Deben Pasar Que</i> . Restricción blanda análoga a nppq pero con peso menor; penaliza sin invalidar la solución.
sbpq	(heredados)	<i>Solo Bien Para Que</i> . Restricción blanda análoga a tppq .
minimizar	(heredados)	Objetivo de minimización: penaliza el máximo valor que toma la consulta en el dominio cuanti-elemento .
maximizar	(heredados)	Objetivo de maximización: penaliza el negativo del mínimo valor de la consulta en el dominio.

1.8. Operaciones

Las expresiones del lenguaje se construyen mediante una jerarquía de nodos de operación. Todos heredan de `operacion-binaria` u `operacion-unaria`,

Cuadro 1.6: Nodos de comprobación de conjuntos.

Nodo	Atributos	Papel
hay-elementos-duplicados	check-duplicados-lista	Comprueba si la lista referenciada por <code>check-duplicados-lista</code> contiene elementos repetidos. Genera código que verifica unicidad.
pertenece	pertenece-elemento pertenece-coleccion	Comprueba si <code>pertenece-elemento</code> es miembro de <code>pertenece-coleccion</code> . Equivale al operador <code>in</code> de Python.

que guardan los operandos.

Los operadores `op-and` y `op-or` son variádicos: aceptan cualquier número de operandos en el DSL y los almacenan como un árbol binario derecho mediante reducción interna. El generador de código los aplana al producir la expresión Python final.

Listing 1.6: Uso combinado de operaciones lógicas, relacionales y aritméticas.

```
;; Condicion compuesta: un profesor esta en un tribunal
;; (disyuncion de cinco igualdades)
(def-op-or (def-op-igual (tutor      (tribunal asig)) prof)
            (def-op-igual (oponente  (tribunal asig)) prof)
            (def-op-igual (presidente (tribunal asig)) prof)
            (def-op-igual (vocal      (tribunal asig)) prof)
            (def-op-igual (secretario (tribunal asig)) prof))

;; Condicion de conflicto: mismo dia y hora, pero tribunales distintos
(def-op-and (def-op-igual (fecha a1) (fecha a2))
            (def-op-igual (momento a1) (momento a2))
            (def-op-distinto (tribunal a1) (tribunal a2)))

;; Expresion aritmetica en una restriccion
(def-op-mayor conflictos-de-horario 0)
```

El código Python generado a partir de estas expresiones es:

Cuadro 1.7: Nodos de operación.

Nodo	Atributos	Papel
operacion-binaria	op-izq op-der	Clase base de todas las operaciones con dos operandos. <code>op-izq</code> y <code>op-der</code> son nodos AST.
operacion-unaria	op-operando	Clase base de las operaciones con un único operando.
<i>Operaciones aritméticas (heredan de <code>operacion-binaria</code>)</i>		
op-suma	(heredados)	Suma aritmética (+).
op-resta	(heredados)	Resta aritmética (-).
op-multiplicacion	(heredados)	Multiplicación (*).
op-division	(heredados)	División (/).
<i>Comprobaciones aritméticas (heredan de <code>operacion-binaria</code>)</i>		
op-igual	(heredados)	Igualdad (==).
op-distinto	(heredados)	Diferencia (!=).
op-mayor	(heredados)	Mayor estricto (>).
op-menor	(heredados)	Menor estricto (<).
op-mayor-igual	(heredados)	Mayor o igual (>=).
op-menor-igual	(heredados)	Menor o igual (<=).
<i>Operaciones lógicas</i>		
op-and	(heredados, variádico)	Conjunción lógica (<code>and</code>). El constructor acepta <i>n</i> operandos y los encadena por reducción.
op-or	(heredados, variádico)	Disyunción lógica (<code>or</code>). Ídem variádico.
op-not	op-operando	Negación lógica (<code>not</code>).

Listing 1.7: Expresiones Python generadas por el compilador del DST.

```
# (def-op-or ...) sobre cinco igualdades
(asig.tribunal.tutor = prof or asig.tribunal.oponente = prof or
 asig.tribunal.presidente = prof or asig.tribunal.vocal = prof or
```

```

    asig.tribunal.secretario == prof)

# (def-op-and ...) de condiciones de conflicto
(a1.fecha == a2.fecha and a1.momento == a2.momento
 and a1.tribunal != a2.tribunal)

# (def-op-mayor consulta 0)
(conflictos_de_horario(horario) > 0)

```

1.9. Resumen de la jerarquía del AST

La tabla 1.8 recoge todos los nodos del AST con su clase padre directa, sus atributos propios y su nivel en la jerarquía de herencia.

Cuadro 1.8: Jerarquía completa de nodos del AST.

Nodo	Padre directo	Atributos propios
nodo-ast	—	(ninguno)
ref-gen-entidad	nodo-ast	nombre-entidad
ref-especifica-ent	nodo-ast	(generados por def-entidad)
comb-entidades	nodo-ast	comb-entidades-lista
lista	nodo-ast	lista-elementos
acceso-a-atributo-de-entidad	nodo-ast	acceso-atributo, acceso-entidad
acceso-a-elemento-de-comb	nodo-ast	acceso-comb-entidad, acceso-comb-origen
consulta-simple	nodo-ast	consulta-args, consulta-var-iter, consulta-dominio, consulta-comprobacion, consulta-operacion, consulta-retorno
hay-elementos-duplicados	nodo-ast	check-duplicados-lista
pertenece	nodo-ast	pertenece-elemento, pertenece-coleccion
operacion-binaria	nodo-ast	op-izq, op-der
operacion-unaria	nodo-ast	op-operando
operacion-aritmetica	operacion-binaria	(ninguno)
op-suma	operacion-aritmetica	(ninguno)
op-resta	operacion-aritmetica	(ninguno)
op-multiplicacion	operacion-aritmetica	(ninguno)
op-division	operacion-aritmetica	(ninguno)
comprobacion-aritmetica	operacion-binaria	(ninguno)
op-igual	comprobacion-aritmetica	(ninguno)
op-distinto	comprobacion-aritmetica	(ninguno)
op-mayor	comprobacion-aritmetica	(ninguno)
op-menor	comprobacion-aritmetica	(ninguno)
op-mayor-igual	comprobacion-aritmetica	(ninguno)
op-menor-igual	comprobacion-aritmetica	(ninguno)
operacion-logica-binaria	operacion-binaria	(ninguno)
op-and	operacion-logica-binaria	(variádico)
op-or	operacion-logica-binaria	(variádico)
operacion-logica-unaria	operacion-unaria	(ninguno)
op-not	operacion-logica-unaria	(ninguno)
cuantificador-restriccion	nodo-ast	cuanti-operador, cuanti-comprobacion, cuanti-elemento